Install Runbook — Windows + Arch dual-boot

Print this. Keep it next to the laptop.

Total wall-clock: ~90 min active, ~2 h with waits.

You'll need:

- Ventoy USB (already built + staged via `pnpm i` on the dev machine — whatever drive letter Windows mounts it at, e.g. `E:` or `F:`)
- Wi-Fi SSID + password
- The Bitwarden master password (for later — Bitwarden desktop in Arch)
- A second device (phone) to read this if you can't print
- A safe place to stash the **BitLocker recovery key** (see Phase 1 step 7)

What gets touched:

- **Samsung 512 GB** → EFI + MSR + Windows 160 GB + Arch btrfs (~316 GB)
- **Netac 128 GB** → Arch recovery ISO + 16 GB swap + ext4 for `/var/log` + `/var/cache`

Two things to know before starting:

- **Hostnames are intentionally different**: Windows = `Metis`, Arch = `inspiron`. Your router sees whichever OS is up. This is cosmetic, not a bug (see recovery §D if it bothers you).
- **First Windows boot after Arch install will prompt for the BitLocker recovery key.** Expected — systemd-boot changes PCR values. Phase 1 step 7 stashes the key in Bitwarden.

Phase 0 — BIOS prep (5 min)

1. Plug the Ventoy USB. Power on. Hammer **F2** to enter BIOS setup.
2. Set:
   - **Boot Mode**: UEFI (not Legacy)
   - **Secure Boot**: **Disabled** for the install (systemd-boot isn't signed out of the box). `decisions.md` wants Secure Boot ON long-term; that's a separate later step using `sbctl` to enroll your own keys. Don't block on it now — get the install running first, then re-enable with signed kernel/bootloader after Phase 3 stabilizes.
   - **SATA Operation**: AHCI (should already be; RAID/Intel RST will hide disks from Arch)
   - **Fast Boot**: Disabled (stops it skipping the boot menu)
3. Save & exit. Hammer **F12** at the Dell logo → boot menu → pick the USB (something like "UEFI: SanDisk …" or the WD label).

If the USB isn't in F12: go back to BIOS → Boot Sequence → Add Boot Option, or make sure USB Boot is enabled. Secure Boot being on will also hide it.

Phase 1 — Windows install (~30 min, mostly unattended)

1. Ventoy menu appears. Pick `Win11_25H2_English_x64_v2.iso`.
2. If Ventoy shows "Boot in normal mode" / "Boot in grub2 mode" — pick **normal**.
3. `autounattend.xml` takes over. You'll see:

- diskpart scrolls, Samsung gets wiped and laid out
- Windows files copy (~10 min)
- Reboots itself **twice** — leave the USB in both times; Ventoy will re-show the menu. **Don't press anything** — it'll auto-select the Win11 ISO via the `autosel` timer. If the timer expires and you're stuck, just re-pick Win11.
- Finally lands on the `Tom` desktop. No prompts, no OOBE.

If you see `No unique 500-600 GB disk found - refusing to proceed`: The inline PowerShell safety check in `autounattend.xml` aborted because either zero or multiple disks fell in the 500–600 GB window. Likely causes and fixes, in order of likelihood:

- **An external drive is plugged in that's also 500–600 GB** → unplug it, press any key to dismiss the pause, power-cycle (hold power), retry F12 → USB.
- **The Samsung is missing from BIOS** → reboot into BIOS (F2). Confirm the Samsung SSD 840 PRO appears under Storage. If it doesn't: reseat the drive, or the SATA controller is still in RAID mode (flip to AHCI — Phase 0 step 2).
- **The Samsung's size is outside the window** → If you're running on different hardware than decisions.md §Q9 describes, the hardcoded 500–600 GB bounds are wrong. Fix: edit `autounattend.xml` — search for `500GB` and `600GB`, widen the window, re-stage the USB with `pnpm stage`.

4. **Wi-Fi should already be connected** — `autounattend.xml` embeds three profiles (`ATTgs5BwGZ`, `rhombus`, `rhombus_legacy`), all WPA2PSK, `connectionMode=auto`. Windows adds all three and connects to whichever is in range. Confirm via the taskbar network icon. If none are visible (different location), click the icon → pick SSID → enter password. The winget-import scheduled task waits up to 20 min for `cdn.w`

`inget.microsoft.com` and proceeds as soon as you're online.

5. Watch for winget to run:
   - It's a scheduled task called **`WingetImportOnce`**, starts ~2 min after logon.
   - Open Task Manager → Details, you'll see `winget.exe` churning. Takes 10–20 min depending on your connection.
   - It self-deletes the task on success.

If winget never runs or fails:

- Open PowerShell, run `schtasks /Query /TN WingetImportOnce /V /FO LIST` — look at **Last Result** (0 = ok, anything else = failed).
- Log files at `C:\Windows\Setup\Scripts\winget-*.log`.
- Manual fallback: `winget import -i C:\Windows\Setup\Scripts\winget-import.json --accept-package-agreements --accept-source-agreements --ignore-versions`

6. **BitLocker runs in parallel with winget** — it's in `FirstLogon.ps1`. With UsedSpaceOnly on a ~10-GB-used fresh install, encryption finishes in ~5 min. Check progress: `manage-bde -status C:` in PowerShell.

7. **Stash the BitLocker recovery key RIGHT NOW** — before doing anything else:
   - File #1: `C:\Windows\Setup\Scripts\BitLocker-Recovery.txt`
   - File #2: `<VENTOY_USB>:\bitlocker-recovery.txt` (same content)
   - Open it. Copy the 48-digit key.

- Paste into Bitwarden as a **Secure Note** titled "Inspiron BitLocker recovery". Save.
- Also take a phone photo of it as a belt-and-suspenders backup.
- Once you've confirmed the key is safely in Bitwarden, delete both files: `Remove-Item C:\Windows\Setup\Scripts\BitLocker-Recovery.txt, <USB_LETTER>:\bitlocker-recovery.txt -Force` (substitute whatever drive letter Windows assigned the Ventoy stick — check **This PC**)
- **Do not skip this.** You'll need this key in Phase 2e — the first Windows boot after Arch install will almost certainly prompt for it.

8. Once winget is done (or even while it's running — Arch install doesn't care about Windows state), shut down. Leave the USB in.

Phase 2 — Arch install (~40 min)

2a. Boot the Arch live environment

1. Power on, F12, pick USB again.
2. Ventoy menu → pick `archlinux-x86_64.iso` → "Boot in normal mode". (The filename is always the undated symlink — `fetch-assets.ps1` mirrors that, not a dated release.)
3. At the Arch ISO menu, pick **"Arch Linux install medium (x86_64, UEFI)"**. You land at a root shell.

2b. Network

- **Ethernet via USB-C dock is the primary path** — plug it in before booting. `install.sh` pings `archlinux.org`; if ethernet is up, it skips Wi-Fi entirely.

- **Wi-Fi fallback**: if no ethernet, `install.sh` auto-connects using the embedded profiles (`ATTgs5BwGZ`, `rhombus`, `rhombus_legacy`). You do nothing.

If BOTH fail (dock ethernet driver missing from live ISO, all Wi-Fi networks out of range), fall back to manual iwctl:

```
iwctl
device list                                # note you
r wifi device, probably wlan0
station wlan0 scan
station wlan0 get-networks
station wlan0 connect YOUR_SSID            # prompts
for passphrase
exit
ping -c2 archlinux.org
```

If `device list` is empty: `rfkill unblock all`, try again. If still empty, plug in the USB-C dock for wired ethernet.

2c. Mount the Ventoy USB data partition

The install script needs access to the Ventoy USB (it reads the chroot script, the Arch ISO for the recovery partition, and the phase-3 files off it).

```
mkdir -p /mnt/ventoy
mount /dev/disk/by-label/Ventoy /mnt/ventoy
ls /mnt/ventoy/phase-2-arch-install/     # sanity:
you should see install.sh + chroot.sh
```

If `by-label/Ventoy` doesn't exist: find it manually: `lsblk -f` — look for the exFAT/NTFS partition on the USB (biggest, not the small 32 MB one). Mount that partition directly: `mount /dev/sdX2 /mnt/ventoy`.

2d. Run the installer

Optional but worth the 20 seconds if your connection feels slow: the live ISO ships with a stale/geographically-random mirrorlist. pacstrap on a bad mirror can appear to hang at <50 KB/s for 20+ min before it finishes. Refresh first:

```
reflector --latest 10 --sort rate --protocol https
--save /etc/pacman.d/mirrorlist
```

If `reflector` complains about no network, you haven't connected yet — go back to 2b. If it exits OK but the result still looks wrong (`head /etc/pacman.d/mirrorlist`), skip it — pacstrap will still work on the default list, just slower.

```
bash /mnt/ventoy/phase-2-arch-install/install.sh
```

It will:

1. Size-detect the Samsung (500–600 GB) and Netac (100–150 GB). **Aborts loudly if either is missing** — don't blindly retry, fix it.
2. Show the plan and ask `[yes/NO]`. Type `yes` exactly.
3. **Prompt once up-front for root password + tom's password** (both confirmed twice; SHA-512-hashed via `openssl passwd -6` before pacstrap starts). Tom's is your daily login — make it strong; you'll bypass it with TPM-PIN and fingerprint later. Nothing else will ask you for input until it's done.
4. Partition, mkfs, pacstrap (~15 min — biggest wait, fully unattended).
5. Enter chroot (passwords handed in via a mode-600 file), install systemd-boot, wire services, write PAM for fingerprint-sudo + gnome-keyring.

6. `dd` the Arch ISO onto the Netac recovery partition (~1 min).
7. Copy `postinstall.sh` + dotfiles into `/home/tom/`.
8. Unmount and print "Done."

If install.sh dies:

- **"No 500–600 GB disk detected"** → phase-1 Windows install didn't run. Boot back into Windows first.
- **"Samsung has <3 partitions"** → same — Windows didn't lay out the disk.
- **pacstrap fails** → network dropped. Reconnect wifi (`iwctl`), re-run the script; it's **not idempotent**, so first: `umount -R /mnt; swapoff -a; wipefs -a /dev/SAMSUNG_new_partition` and start over. (Recovery path is ugly because it's a fresh install — if you're uncertain, just start from scratch, it's fast.)
- **"Could not find EFI System partition"** → Windows install put the EFI somewhere weird. Use `gdisk -l /dev/sda` to confirm GUID `c12a7328…`.
- **chroot script fails mid-way** → you can `arch-chroot /mnt` manually and finish the remaining steps from `chroot.sh` by hand.

9. When it says "Done": `reboot`. Pull the USB as it powers off.

2e. First boot into Arch

- systemd-boot menu appears with **Arch Linux** + **Arch Linux (fallback)** + auto-discovered **Windows Boot Manager**. 3-sec timeout; Arch is default.
- It boots to a black screen, then to **SDDM** (graphical login).

If the systemd-boot menu doesn't show Windows: boot into Arch anyway, then as root: `bootctl list` should show

Windows. If not: `ls /boot/EFI/Microsoft/Boot/bootmgfw.efi` — missing file means Windows install didn't write it (rare). Fix later; don't block on this.

Expected: BitLocker recovery prompt on first Windows boot after this step. Systemd-boot installing itself to the shared EFI changes the TPM's PCR values → BitLocker can't auto-unseal → you get the blue "Enter recovery key" screen. This is not a bug.

- Type the 48-digit key from Bitwarden.
- Windows unlocks, boots normally.
- Windows re-seals to the new PCR values automatically. Next boots: silent unlock, no prompt ever again (unless bootloader changes).
- **If you don't have the key when this screen appears — you are locked out of Windows.** Phase 1 step 7 exists for this reason.

If nothing boots: BIOS → Boot Sequence → make sure "Linux Boot Manager" is listed and first. If only "Windows Boot Manager" is there, run from a live USB: `bootctl --path=/boot install` to re-register.

2f. Clear the BitLocker prompt now (before Phase 3)

Do this before starting Phase 3 — you want the BitLocker key mess handled while you're still thinking about it, not 25 minutes into Hyprland setup.

1. Reboot. At the systemd-boot menu, pick **Windows Boot Manager**.
2. BitLocker prompts for the recovery key. Type the 48-digit key from Bitwarden ("Inspiron BitLocker recovery").
3. Windows boots. Log in as `Tom`. Let it sit for 30 seconds so BitLocker re-seals against the new PCR values.

4. Shut down (don't just log out — Fast Startup is off, but a clean shutdown is still cleaner). Start → Power → Shut down, or from an **elevated** PowerShell (right-click → Run as administrator) run `shutdown /s /t 0`. A non-elevated `shutdown` call fails silently with "Access is denied" on stock Windows 11 unless the `Tom` account is in the local Administrators group (it is, per autounattend — so a normal PowerShell works too; the elevation path is the belt-and-suspenders option).
5. Power on → systemd-boot → pick **Arch Linux** → back into Arch. No more BitLocker prompts from here.

If the BitLocker key doesn't work: you have the phone photo from Phase 1 step 7 as a last-ditch copy. If even that doesn't work, Windows is gone and you need to reinstall — Arch is unaffected.

Phase 3 — Post-install (~25 min)

3a. Get a terminal

At SDDM, log in as **tom**. Hyprland starts with **no config yet** → blank screen, no keybindings, no launcher. That's expected.

Switch to a TTY: `Ctrl + Alt + F2` → text login as `tom`.

3b. Run postinstall

```
chmod +x ~/postinstall.sh
~/postinstall.sh
```

It will:

1. `sudo` prompt for your password — type it. (No fingerprint/PIN yet.)

2. **pacman** — all CLI tooling, Bitwarden (desktop + CLI), Ghostty, fuzzel, cliphist, swaync, satty, hyprshot, mise, chezmoi, gh, snapper. Signed binaries from `extra`, ~5 min.
3. **yay** — only the 4 AUR-exclusive packages: `visual-studio-code-bin`, `microsoft-edge-stable-bin`, `catppuccin-sddm-theme-mocha`, `pinpam-git`. ~5 min build time.
4. Installs Claude Code CLI: `mise use -g node@lts` pulls a LTS Node via mise, then `mise exec -- npm install -g @anthropic-ai/claude-code`. (There is no mise plugin named "claude-code" — Claude Code is shipped through npm. Completions are printed at runtime by `claude --print-completion zsh`; no external file download.) **No local SSH keys are generated** — keys live in your Bitwarden vault as "SSH key" items and surface via `~/.bitwarden-ssh-agent.sock` once Bitwarden desktop is running with the SSH-agent toggle on (Phase 3e). A planter in `~/.zshrc.d/arch-ssh-signing.zsh` waits until `ssh-add -L` returns a pubkey, then wires git commit signing + registers the key with GitHub automatically.
5. **Points Bitwarden at your self-hosted server** `https://hass4150.duckdns.org:7277` — CLI via `bw config server`, desktop via pre-seeded `~/.config/Bitwarden/data.json`. If the desktop login screen still shows "bitwarden.com" in the "Logging in on:" dropdown, pick **Self-hosted** and paste that URL manually.
6. **Prompt you to enroll your fingerprint** — touch the sensor 5 times, slight re-position each touch. Reader is auto-detected; if enrollment fails the script prints a diagnostic (full `lsusb`, `fprintd-list`, last 20 log lines) and offers to install `libfprint-git` from AUR and retry.
7. **Prompt you to set a TPM-PIN** (`pinutil setup`) — 6+ chars. This is what you'll type for sudo from now on.

8. Wires `~/.ssh/config` for Bitwarden SSH agent.
9. Plants `~/.zshrc.d/arch-first-login.zsh` (one-shot: `bw login` + `gh auth login` + git name/email) and `~/.zshrc.d/arch-ssh-signing.zsh` (every-login, self-deletes once SSH signing is wired).
0. Builds zgenom plugins (warms cache so first login is fast), writes tmux/helix/ghostty configs.
1. Takes a **snapper baseline snapshot** of `/` — you can roll back later via `snapper -c root list`.
2. Installs USB-serial udev rules (ESP32/Pico/FTDI/CH340) and adds you to `uucp`.
3. Runs the **end-4/dots-hyprland installer interactively** — it'll ask questions. Accept defaults unless you know better. If it asks whether to **overwrite existing config files**, say **yes** — your `~/.config` is fresh and there is nothing here worth keeping. If it asks about a user-level systemd service restart, say yes too.
4. Prints a verify table — scan for **FAIL** rows.

If fingerprint enroll fails: postinstall already handles the Goodix fallback interactively. If you declined or it still fails:

```
SKIP_FPRINT=1 ~/postinstall.sh    # re-run skipping
finger step
```

Then manually: `lsusb | grep -iE 'goodix|validity|synaptics'`, `fprintd-list-devices`. Check https://fprint.freedesktop.org/supported-devices.html for your VID:PID.

If `pinutil` is missing after AUR install:

```
SKIP_PIN=1 ~/postinstall.sh       # re-run skipping
PIN step
# Then manually:
yay -S pinpam-git
sudo pinutil setup
```

If something else fails mid-run: re-run `~/postinstall.sh`. It's idempotent.

3c. Reboot into Hyprland

```
sudo reboot
```

- SDDM again. Log in as tom.
- Hyprland comes up with end-4 config: waybar at top, wallpaper, keybindings live.
- Default terminal: `Super + Return` opens Ghostty (check end-4 keybind cheatsheet — it's `Super+T` in some variants).
- App launcher: `Super + Space` (fuzzel).

Escape hatch if NO keybinding opens a terminal (keybind variant mismatch, Hyprland config didn't install, wayland handshake fail): `Ctrl + Alt + F3` drops to TTY3 where you can log in as `tom` and debug. `Ctrl + Alt + F1` jumps back to the SDDM/Hyprland session. Worst case: from TTY3, `journalctl --user -u hyprland -b` tells you why Hyprland dropped you to a featureless compositor.

3e. Bitwarden one-time setup

1. Launch Bitwarden from the launcher.
2. Log in with your master password (only time you type it).
3. **Settings → Security → enable "Unlock with system keyring"**. You'll be asked for the master password once

more — after that, gnome-keyring holds it and Bitwarden auto-unlocks at login.

4. **Settings** → **SSH agent** → **Enable**. The socket appears at `~/.bitwarden-ssh-agent.sock` (already wired into `~/.ssh/config`).
5. Add any SSH keys you want as **"SSH key"** vault items.
6. **You MUST log out and back in now** (or at least close + reopen every terminal). `SSH_AUTH_SOCK` is set by `~/.zshrc.d/bitwarden-ssh-agent.zsh` at shell start — it only notices the new socket in a fresh shell. Skip this and the 3f sanity check `ssh-add -l` will fail even though the agent is actually working.

After the re-login: Bitwarden auto-unlocks (via gnome-keyring), and `ssh-add -l` lists your keys with no prompt.

3f. Sanity checks

```
# sudo: clear cache, then test. Pay attention to WHICH prompt appears.
sudo -k
sudo echo ok
```

The prompt you see tells you what's wired:

- `PIN:` → pinpam is working. TPM-PIN is your daily sudo auth. ✓
- `Place your finger on the fingerprint reader` → fprintd is working. ✓
- `[sudo] password for tom:` → **neither TPM-PIN nor fingerprint is wired**. Something went wrong. See recovery §I below before relying on this.

```
# fingerprint enrolled?
fprintd-list tom                # should show a fing
erprint enrolled

# PAM wiring
sudo grep -E 'pam_pinpam|pam_fprintd' /etc/pam.d/su
do
# Expect both lines, both marked `sufficient`, befo
re the pam_unix.so fallback.

# ssh agent
echo $SSH_AUTH_SOCK             # should be ~/.bitwa
rden-ssh-agent.sock (set by ~/.zshrc.d/bitwarden-ss
h-agent.zsh)
ssh-add -l                      # should list keys o
nce Bitwarden desktop is unlocked + SSH-agent toggl
e is on
```

Phase 4 — Start the Claude session with the handoff

1. Open Ghostty. Start tmux: `tmux new -s main` (prefix = **Ctrl+a**).
2. Clone the setup repo so Claude has context:

   ```
   mkdir -p ~/src
   cd ~/src
   # Primary: clone from GitHub (if repo is pushe
   d).
   git clone git@github.com:fnrhombus/arch-setup.gi
   t arch-setup@fnrhombus
   cd arch-setup@fnrhombus
   ```

   If the clone fails (repo not pushed, or gh auth not wired yet), recover from the Ventoy USB — it still has the repo contents synced there pre-install:

```
# Re-insert the Ventoy USB (you pulled it at end
of Phase 2d).
sudo mkdir -p /mnt/ventoy
sudo mount /dev/disk/by-label/Ventoy /mnt/ventoy

# Sanity-check what's actually at the USB root b
efore copying - the sync
# step (pre-install) may have only pushed the ph
ase-2/3 script dirs and
# not the markdown docs. If the `.md` files are
missing, Claude still has
# enough to proceed with just phase-3 script + C
LAUDE.md.
ls /mnt/ventoy/*.md /mnt/ventoy/CLAUDE.md 2>/de
v/null

mkdir -p ~/src/arch-setup@fnrhombus
# Copy every repo file that IS on the USB; missi
ng ones silently skip
# thanks to 2>/dev/null. Falls through to a whol
e-USB snapshot only if
# the targeted copy produced no files at all.
cp -r /mnt/ventoy/phase-2-arch-install \
      /mnt/ventoy/phase-3-arch-postinstall \
      /mnt/ventoy/INSTALL-RUNBOOK.md \
      /mnt/ventoy/autounattend.xml \
      /mnt/ventoy/decisions.md \
      /mnt/ventoy/handoff.md \
      /mnt/ventoy/CLAUDE.md \
      ~/src/arch-setup@fnrhombus/ 2>/dev/null
if [[ -z "$(ls -A ~/src/arch-setup@fnrhombus 2>/
dev/null)" ]]; then
    warn "Targeted copy got nothing - falling ba
ck to a whole-USB snapshot."
    cp -r /mnt/ventoy ~/src/arch-setup-snapshot
    cd ~/src/arch-setup-snapshot
else
    cd ~/src/arch-setup@fnrhombus
fi
```

3. Start Claude in that directory:

```
claude
```

4. Feed it the handoff as the first message:

```
Read handoff.md. That's the brief. Start by walk
ing me through the
Hyprland keybindings — I'm looking at a fresh Hy
prland session with
the end-4/illogical-impulse dotfiles and I don't
know how to do
anything yet.
```

   Claude will read `handoff.md`, `decisions.md`, and `CLAUDE.md` on its own and have the full context.

Things that will eat time but aren't broken

- **pacstrap and yay builds look stuck** — they're not. `base-devel` pulls ~50 packages, AUR builds compile from source. Don't Ctrl-C.
- **zgenom first-login rebuild** — if postinstall's pre-build didn't fully warm the cache, your first `zsh` login will take 20 seconds. One-time.
- **end-4 installer** — asks a dozen questions. Defaults are fine.
- **fingerprint enroll** — takes 5 touches but the reader is picky about angle. If it keeps rejecting, lift fully off between touches.

Things that are actually broken if you see them

- **"No network" during pacstrap** → wifi dropped. Reconnect, restart install.
- **systemd-boot doesn't list any entries after `bootctl list`** → reinstall with `bootctl --path=/boot install` as root.

- **SDDM loops back to login** → Hyprland is crashing. Drop to TTY, `cat ~/.local/share/hyprland/hyprland.log`. Usually a missing config include from end-4 — re-run its installer.
- **`sudo` rejects your TPM-PIN** → TPM got reset or the PIN index was evicted. Re-run `sudo pinutil setup`. Password still works as fallback.

First-boot failure modes (will it all actually come up?)

These are the failure modes most likely to hit on the very first boot after `install.sh` reboots. Each has a recovery path that assumes you have the Ventoy USB handy.

A. Kernel panic: "Cannot open root device" / "VFS: Unable to mount root fs"

Cause: btrfs module didn't make it into the initramfs, so the kernel boots but can't read `/`. `chroot.sh` now forces `MODULES=(btrfs)` in `/etc/mkinitcpio.conf` — but if that line got overridden or mkinitcpio threw a warning you ignored, this is what you see.

Fix: Boot the Ventoy USB → Arch ISO → live environment. Then:

```
mount -o subvol=@ /dev/disk/by-label/ArchRoot /mnt
mount /dev/disk/by-partlabel/EFI\ system\ partition
/mnt/boot 2>/dev/null \
  || mount "$(blkid -o device -t PARTLABEL='EFI Sys
tem Partition')" /mnt/boot
arch-chroot /mnt
grep -q '^MODULES=(btrfs)' /etc/mkinitcpio.conf ||
sed -i 's/^MODULES=.*/MODULES=(btrfs)/' /etc/mkinit
cpio.conf
mkinitcpio -P
exit
reboot
```

B. Emergency shell at boot: "Dependency failed for /var/log"

Cause: The `/var/log` and `/var/cache` bind mounts race the ext4 mount at `/mnt/netac-var`. `install.sh` now writes `x-systemd.requires-mounts-for=/mnt/netac-var` into both fstab entries, but if the fstab-rewrite step silently skipped (look for `[!] fstab post-process failed` in install output), the ordering won't be there.

Fix (from the emergency shell at boot):

```
# Log in as root (password you set in chroot).
mount -o remount,rw /
vim /etc/fstab   # or `vi` - nano is NOT installed,
only vim and helix are pacstrapped
# Find the two lines whose target is /var/log and /
var/cache.
# In their options column (4th column), append:
#   ,x-systemd.requires-mounts-for=/mnt/netac-var
# Save with :wq
systemctl daemon-reload
reboot
```

If editing fstab from emergency mode is scary: add `systemd.unit=rescue.target` to the kernel command line in systemd-

boot (press `e` on the Arch entry at boot, append to `options=`), which gives a full single-user shell with `/` writable.

C. BitLocker recovery prompt on next Windows boot

This is expected, not a failure. systemd-boot installing itself to the shared EFI rewrites the PCR values the TPM sealed BitLocker against. First Windows boot after Arch install → blue "Enter recovery key" screen.

Fix: Type the 48-digit key (stored in Bitwarden as "Inspiron BitLocker recovery" per Phase 1 step 7). Windows unseals, re-seals to the new PCRs. Never prompts again unless the bootloader changes.

D. Hostname shows as `Metis` on the network, not `inspiron`

Cause: Windows unattend names the machine `Metis`, Arch's `chroot.sh` names it `inspiron`. Both are correct for their respective OS — they just don't match. Your router's DHCP lease will show whichever OS booted most recently.

This is cosmetic. If you care: edit `/etc/hostname` in Arch to `Metis` (and the `127.0.1.1` line in `/etc/hosts`), or change Windows via `Rename-Computer -NewName inspiron -Restart` from an elevated PowerShell. Neither change is required for anything to work.

E. Snapper complains `config "root" does not exist` or `.snapshots not found`

Cause: `postinstall.sh` skips `snapper -c root create-config /` (which would try to re-create an already-mounted `/.snapshots` subvolume) and writes the config by hand. If that hand-write was skipped — e.g., `postinstall.sh` was re-run after partial state — you're missing the config.

Fix:

```
sudo install -d -m 750 /etc/snapper/configs
sudo cp /etc/snapper/config-templates/default /etc/
snapper/configs/root
sudo sed -i 's|^SUBVOLUME=.*|SUBVOLUME="/"|;s|^ALLO
W_USERS=.*|ALLOW_USERS="tom"|' /etc/snapper/config
s/root
# Guard against double-append if you're re-running
this:
grep -q '^SNAPPER_CONFIGS=.*root' /etc/conf.d/snapp
er 2>/dev/null || \
  echo 'SNAPPER_CONFIGS="root"' | sudo tee -a /etc/
conf.d/snapper
sudo chown :tom /.snapshots 2>/dev/null || true
sudo chmod 750 /.snapshots
sudo snapper -c root create --description "manual b
aseline"
```

F. Bluetooth doesn't auto-enable after reboot

Cause: The AutoEnable sed rewriters in `chroot.sh` all silently no-op'd (a distro change in `/etc/bluetooth/main.conf` defaults would do it). There's now a final `>>` append fallback, but if the config file didn't exist at all when chroot.sh ran, the whole `if` block was skipped.

Fix:

```
sudo tee -a /etc/bluetooth/main.conf >/dev/null <<E
OF

[Policy]
AutoEnable=true
EOF
sudo systemctl restart bluetooth
```

G. Can't get back into Windows from the Arch systemd-boot menu

Cause: Windows Boot Manager didn't auto-register with systemd-boot. Check `bootctl list` — if you don't see `title: Windows Boot Manager`, the `\EFI\Microsoft\Boot\bootmgfw.efi` file is missing or wasn't detected.

Fix: Reboot, hit F12 at the Dell logo, pick "Windows Boot Manager" directly from the firmware menu. From inside Windows, re-run Arch's `bootctl` from a live USB later to re-register. You are not stuck — both OSes are still bootable via F12.

H. SDDM shows but fingerprint prompt never appears at login

SDDM login doesn't use fingerprint by default — it's wired for **sudo/polkit/hyprlock** only. Type your password at SDDM. Fingerprint kicks in the first time you `sudo` after login.

I. `sudo` fails with "PAM module not found" — you're locked out of root

Cause: If `pinpam-git` failed to install but `postinstall.sh`'s PAM wiring ran anyway (or vice versa), `/etc/pam.d/sudo` references a `pam_pinpam.so` module that doesn't exist on disk. Same for `pam_fprintd.so` if fprintd was uninstalled. PAM's error mode for a missing module is to **fail the entire auth stack**, not fall through — which means your password won't work either. You are hard-locked.

Fix — from the Ventoy USB Arch live environment:

```
# Boot Ventoy → Arch ISO → live env → connect wifi.
mount -o subvol=@ /dev/disk/by-label/ArchRoot /mnt
arch-chroot /mnt

# Strip every non-standard auth line from /etc/pam.
d/sudo back to defaults.
cat > /etc/pam.d/sudo <<'EOF'
#%PAM-1.0
auth       include      system-auth
account    include      system-auth
session    include      system-auth
EOF
exit
reboot
```

After reboot, `sudo` takes your **password** (TPM-PIN + fingerprint are gone until you re-wire PAM). Fix the missing module: reinstall `pinpam-git` or `fprintd`, then re-run the PAM wiring blocks from `postinstall.sh` by hand.

Prevention: after `postinstall.sh` finishes, always test sudo from a **second** terminal before closing the one you ran the script in. If sudo fails, you can fix PAM from the still-privileged parent session without rebooting.

J. `ssh-add -l` returns "error fetching identities" in every new shell

Cause: Bitwarden desktop isn't running, OR Bitwarden's SSH-agent toggle is off, OR the socket moved. `~/.zshrc.d/bitwarden-ssh-agent.zsh` only exports `SSH_AUTH_SOCK` if the socket file exists, so if the socket is gone, `ssh-add` talks to no agent and errors.

Fix:

1. Check the socket exists: `ls -l ~/.bitwarden-ssh-agent.sock`

2. If missing: launch Bitwarden desktop, unlock, Settings → SSH agent → toggle on. Quit and reopen a terminal.
3. If present but `ssh-add -l` still errors: `SSH_AUTH_SOCK=~/.bitwarden-ssh-agent.sock ssh-add -l` to rule out an env issue.
4. If the SSH-signing planter at `~/.zshrc.d/arch-ssh-signing.zsh` was never deleted, it will try again on each login — once Bitwarden is back online with at least one "SSH key" vault item, the planter fires and self-deletes.

K. "Login incorrect" at SDDM / first TTY — you fat-fingered `tom`'s password during chroot

Cause: `chroot.sh` calls `passwd tom` interactively. If you mistyped both times (the `until passwd tom; do :; done` loop only retries on non-matching password confirmation, NOT on wrong-but-matching typos), you've set a password you can't reproduce. Same goes for `root`.

Fix — from the Ventoy USB Arch live environment:

```
# Boot Ventoy → Arch ISO → live env.
mount -o subvol=@ /dev/disk/by-label/ArchRoot /mnt
arch-chroot /mnt
passwd tom        # set a new one
passwd            # while you're here, reset root t
oo if needed
exit
umount -R /mnt
reboot
```

No other state needs resetting — PAM, keyring, and Bitwarden are all keyed off the new password automatically from the next login.

Recovery

- **Primary recovery**: boot the Ventoy USB, pick the Arch ISO → live environment with `pacstrap`, `arch-chroot`, etc.
- **Secondary recovery**: the Netac has the Arch ISO dd'd onto partition 1. It's not auto-discovered by systemd-boot (systemd-boot can't chain-load a raw ISO partition). To use it, boot into Dell F12 boot menu and pick the Netac's EFI entry — the ISO's own bootloader takes over. So: same live environment, without needing the USB.
- **Full reinstall**: boot USB → Arch ISO → re-run `install.sh`. It's size-gated and will abort if anything looks off, so you can't accidentally double-wipe.

Package-name drift (AUR)

If any `yay -S` call in `postinstall.sh` fails with "package not found", the AUR name changed. Search with `yay -Ss <partial>` and substitute. The likely-drifters:

- `bitwarden` (sometimes `bitwarden-desktop`)
- `bitwarden-cli` (stable)
- `pinpam-git` (could be `pinpam` if it ever reaches stable)
- `catppuccin-sddm-theme-mocha` (rename-prone)
- `hyprshot` (check `hyprshot-git` from AUR if the stable name disappears)

Re-run `postinstall.sh` after fixing — it's idempotent.